SOFTWARE REQUIREMENTS SPECIFICATION

" CHAINSHIELD TRANSPORTATION SYSTEM "

Resilient Logistics & Dynamic Supply Chain Optimization

Prepared as per IEEE 830 SRS guidelines

MCA Semester 3, Academic Year 2026-27

To

LJ INSTITUTE OF COMPUTER APPLICATION

Affiliated to LJK University

Guided By:

Prof. Dhaval Shah

Prepared By: (Group No. 59)

Bhavik A. Visani (D51 / 25004401110170)
Diya Shah (D36 / 25003401110017)
Shruti Somani (D42 / 25004401110259)

INDEX

Sr.	Particulars	Page No
1	**Introduction** 1) Purpose 2) Document Scope 3) Definitions, Acronyms & Abbreviations 4) References 5) Overview	3
2	**Overall Description** 1) Product Perspective 2) Product Functions 3) User Classes & Characteristics 4) Operating Environment 5) Design & Implementation Constraints 6) Assumptions & Dependencies	5
3	**System Architecture & Use Cases**	7
4	**Specific Requirements — Functional**	9
5	**External Interface Requirements** 1) User Interfaces 2) Hardware Interfaces 3) Software Interfaces 4) Communication Interfaces	16
6	**Non-Functional Requirements** 1) Performance 2) Security 3) Reliability 4) Usability 5) Portability 6) Maintainability	17
7	**Data Requirements**	18
8	**Appendix — Traceability & Bibliography**	19

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the ChainShield Transportation System, a project developed by Group No. 59 for MCA Semester 3 at LJ Institute of Computer Application. It gives the project guide, examiners, and the development team a single, unambiguous reference for what the system does, following the structure recommended by IEEE Standard 830. This revision (v2) reflects the completed implementation; Section 8 traces every requirement to its code module and test case.

1.2 Document Scope

This document covers the ChainShield Transportation System rebuild — a Flask, SQLite, and Docker-based application that continuously scores active shipments for disruption risk and proposes optimised route adjustments. It does not cover the original FastAPI/React ChainShield prototype built for an earlier solution-building initiative, which remains a separate, already-deployed codebase and is not part of this specification.

1.3 Definitions, Acronyms & Abbreviations

Term	Meaning
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
Disruption	A shipment event whose composite risk score has reached MEDIUM (≥ 50); it is escalated if the score reaches HIGH (≥ 70). AT-RISK (≥ 30) shipments are flagged and watched.
Risk Engine	The scheduled service that computes a 0-100 composite risk score for each active shipment
Route Solution	A ranked alternative (hold, hub reroute, mode switch, or continue) generated for an open disruption; option 1 is the recommendation
Field Operator	A user with role captain, pilot, driver, or loco_pilot who executes shipments
ORM	Object-Relational Mapper (Flask-SQLAlchemy)

1.4 References

- MCA Semester 3 Project Documentation — ChainShield_MCA_Sem3_Documentation.docx (Group 59, 2026)
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- Flask Documentation — https://flask.palletsprojects.com/
- SQLite Documentation — https://www.sqlite.org/docs.html
- Flask-SocketIO Documentation — https://flask-socketio.readthedocs.io/

1.5 Overview

Section 2 describes the product at a high level — its perspective, functions, users, and constraints. Section 3 summarises the system architecture and use cases. Section 4 lists detailed functional requirements, each with a unique identifier for traceability. Section 5 covers external interfaces, Section 6 covers non-functional requirements, Section 7 defines the data requirements, and Section 8 provides a requirements traceability summary and bibliography.

2. OVERALL DESCRIPTION

2.1 Product Perspective

ChainShield Transportation System is a new, self-contained web application — it is not a component of, or dependent on, any other system. It is built as a single Flask application following the app-factory and blueprint patterns, backed by a local SQLite database, and packaged with Docker for portable deployment. It optionally integrates with two external services — the OpenWeatherMap API for live weather signals and the Google Gemini API for its AI assistant — both of which degrade gracefully to simulated data when unavailable, so the system remains fully demonstrable offline.

2.2 Product Functions

- Role-based user registration, authentication, and session management
- Shipment management (create, view, edit, delete) with role-scoped visibility
- Continuous, scheduled disruption-risk scoring for every active shipment
- Automatic disruption detection, escalation, and ranked route-solution generation
- Real-time in-app notification delivery and direct messaging between users
- Administrative management of users, logistics hubs, and cargo types, with an audit log
- A natural-language AI assistant for supply-chain queries
- A live map visualisation of shipments and disruption zones
- Reports with date-range filters and CSV export
- A demo / simulation mode for presentations without live data

2.3 User Classes and Characteristics

User Class	Technical Expertise	Primary Use
Admin	Moderate — comfortable with web forms and configuration	User, hub, and cargo-type management; reports; audit review; system configuration
Manager	Low to moderate — a logistics professional, not a technical user	Managing shipments, monitoring risk, resolving disruptions, reports
Field Operator	Low — expects a simple, mobile-friendly interface	Viewing assigned shipments, updating status, receiving alerts, chat

2.4 Operating Environment

- Server: any OS that supports Docker (Windows, Linux, macOS) running Python 3.10+
- Client: any modern desktop or mobile web browser (Chrome, Edge, Firefox, Safari)
- Database: SQLite, file-based, persisted via a Docker volume
- Deployment: Docker container exposing the application on port 5000

2.5 Design and Implementation Constraints

- Must use Python and Flask, per the confirmed academic technology stack

- Must remain fully independent of the original FastAPI/React ChainShield prototype — no shared code or database
- Must run without requiring paid API keys — external APIs are optional with simulated fallbacks
- Must be containerisable with a single Dockerfile and docker-compose.yml, with no manual server configuration

2.6 Assumptions and Dependencies

- It is assumed the host machine has Docker Desktop (or an equivalent Docker engine) installed for containerised deployment
- It is assumed shipment position and weather data may be simulated for demonstration, in the absence of live GPS/telematics hardware
- The system depends on Flask, Flask-SQLAlchemy, Flask-Login, Flask-SocketIO, Flask-WTF, and APScheduler, as listed in requirements.txt

3. SYSTEM ARCHITECTURE & USE CASES

The system follows a layered architecture: a Flask + Jinja2 + Bootstrap presentation layer (with Leaflet.js and Chart.js) talking to the server over HTTP and WebSockets, core services (Risk Engine, Route Solver, Notifications/Chat, AI Assistant, Admin & Reports) in the middle, a Flask-SQLAlchemy data access layer, and a SQLite database at the base — the entire stack packaged inside a Docker container.

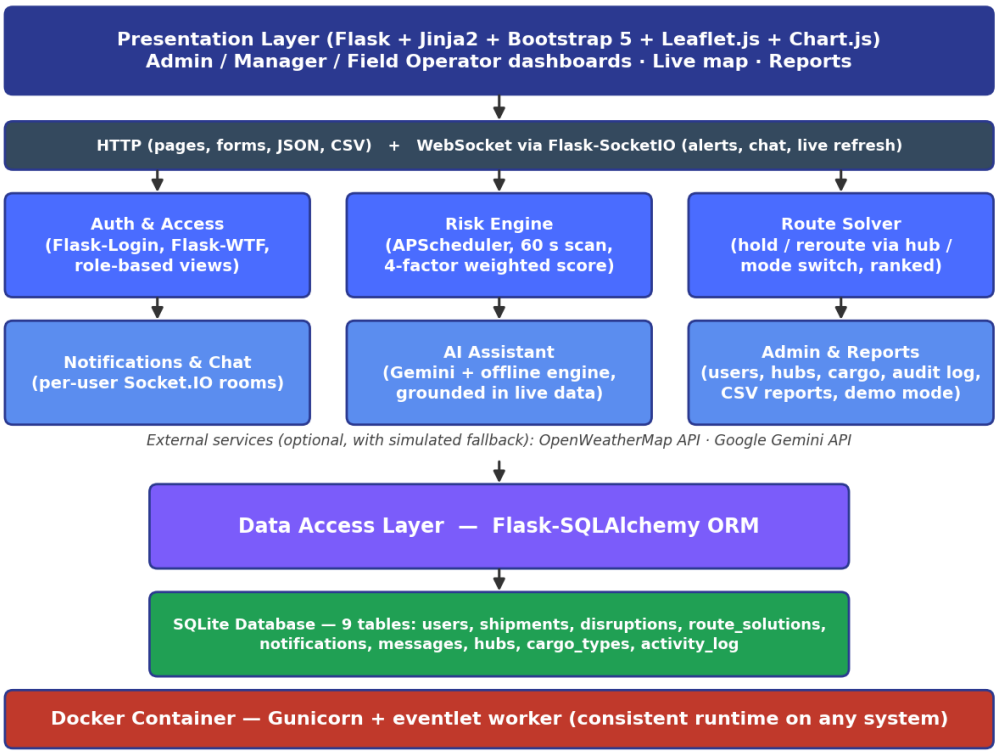


Fig 3.1 — ChainShield System Architecture

Three actors interact with the system — Admin, Manager, and Field Operator — each restricted to the use cases relevant to their role; a background scheduler scores risk automatically, as summarised below.

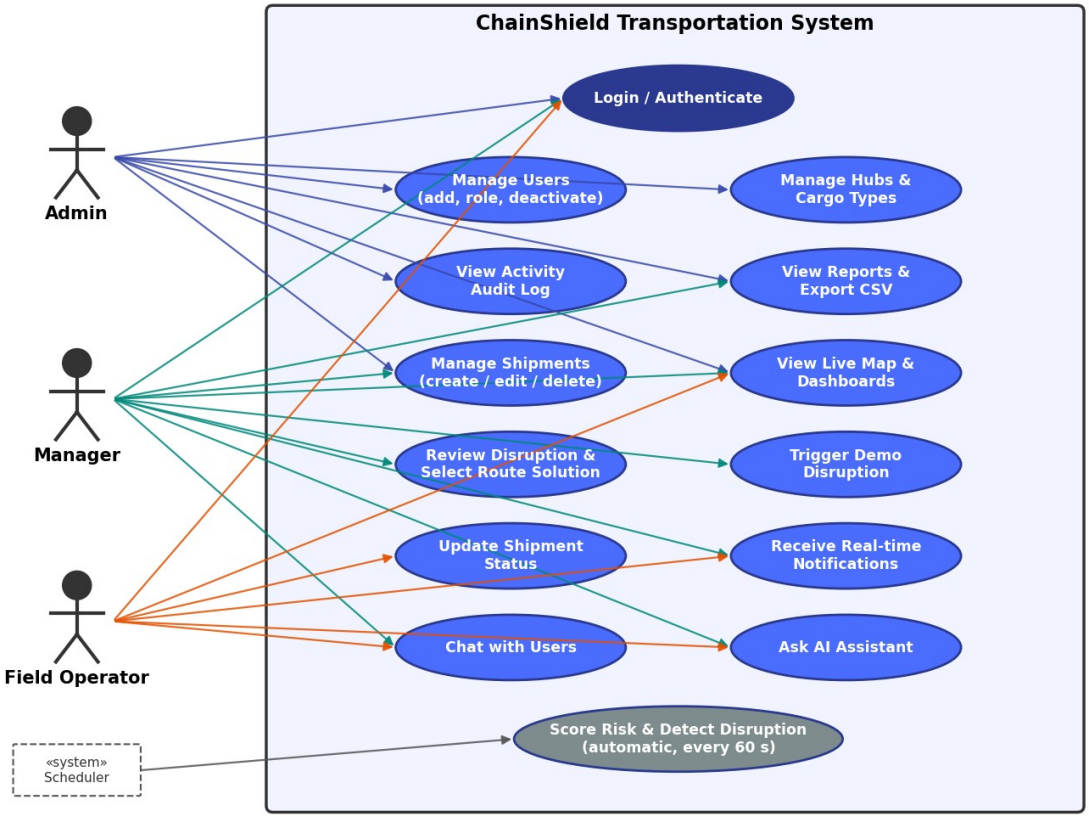


Fig 3.2 — Use Case Diagram

4. SPECIFIC REQUIREMENTS — FUNCTIONAL

Each functional requirement below is uniquely identified (FR-n) for traceability against test cases and the implementation. Requirements are grouped by the module that implements them.

4.1 Authentication & Access Control

FR-1 User Registration

FR-1	User Registration
Description	Allow a new user to create an account with a name, email, password, and role.
Input	Name, email, password (6+ characters), role (manager / captain / pilot / driver / loco_pilot)
Processing	Validate the email is not already registered and the role is allowed (admin cannot be self-assigned — an existing admin grants it); hash the password with Werkzeug; persist the new User row.
Output	New account created; user redirected to the login page with a confirmation message.

FR-2 User Login

FR-2	User Login
Description	Authenticate a registered user by email and password.
Input	Email, password
Processing	Look up the user by email; verify the password hash and that the account is active; on success, start a Flask-Login session and write an audit-log entry.
Output	User is logged in and redirected to their role-specific dashboard; on failure, an error message is shown.

FR-3 User Logout

FR-3	User Logout
Description	End the current user's authenticated session.
Input	Authenticated session
Processing	Clear the Flask-Login session.
Output	User is redirected to the login page.

FR-4 Role-Based Access Control

FR-4	Role-Based Access Control
Description	Restrict each dashboard and admin action to the appropriate user role.
Input	Current user's role, requested route
Processing	Check the requesting user's role on every route (admin-only, manager-or-admin, or assigned operator); field operators only ever see their own shipments and disruptions.
Output	Authorised users see the page; unauthorised users receive 403 Forbidden (or 404 for records outside their scope).

4.2 Risk Engine

FR-5 Risk Score Calculation

FR-5	Risk Score Calculation
Description	Compute a composite 0-100 risk score for a shipment from four weighted signals.
Input	Weather severity, route congestion, cargo sensitivity, historic delay probability (each 0-100)
Processing	score = 0.40×weather + 0.25×congestion + 0.20×cargo + 0.15×history, rounded to 2 decimals.
Output	A composite risk_score and a risk_level classification (LOW / AT-RISK / MEDIUM / HIGH) stored on the shipment.

FR-6 Scheduled Risk Scanning

FR-6	Scheduled Risk Scanning
Description	Automatically re-score every active shipment on a fixed interval.
Input	All shipments with status 'In-transit'; interval from RISK_SCAN_INTERVAL_SECONDS (default 60 s)
Processing	APScheduler triggers a scan job that (in demo mode) moves each shipment toward its destination hub, recalculates its risk score (FR-5) and updates its risk_level; shipments reaching the destination are marked Delivered.
Output	Updated risk scores across all active shipments; disruptions created or escalated (FR-7); a live 'shipments_updated' event refreshes every open map and dashboard.

4.3 Disruption Management

FR-7 Automatic Disruption Detection

FR-7	Automatic Disruption Detection
Description	Create a disruption record automatically when a shipment's risk score reaches MEDIUM (≥ 50), and escalate it when the score reaches HIGH (≥ 70).
Input	Shipment risk_score, risk_level
Processing	If risk_level is MEDIUM or HIGH and no open disruption exists, create a Disruption (dominant factor = highest of the four signals) and trigger FR-8. If one is open and the level has risen, escalate its severity and re-plan its route solutions.
Output	A new or escalated open disruption; real-time notifications to all admins, managers and the assigned field operator.

FR-8 Route Solution Generation

FR-8	Route Solution Generation
Description	Generate ranked route alternatives for an open disruption.
Input	Disruption's shipment, current position, origin/destination hubs, transport mode, risk score
Processing	Build candidates — hold at the nearest hub, continue, reroute via the two hubs with the smallest haversine detour, and a mode switch for the remaining leg — each with extra time, extra cost (₹) and residual risk. Rank by 0.60 × risk + 0.25 × normalised time + 0.15 × normalised cost.
Output	Three to five RouteSolution rows linked to the disruption; option 1 is marked Recommended.

FR-9 Disruption Resolution

FR-9	Disruption Resolution
Description	Allow a manager to select a route solution and resolve the disruption.
Input	Manager's selected option_no
Processing	Mark the chosen RouteSolution as selected; set the disruption to Resolved with resolved_by/resolved_at; put the shipment On-hold if a hold option was chosen; log the decision; notify the assigned field operator.
Output	Disruption closed; field operator notified of the chosen route (FR-10).

4.4 Notifications & Communication

FR-10 In-App Notification Delivery

FR-10	In-App Notification Delivery
Description	Deliver real-time in-app alerts to a specific user.
Input	Target user_id, message text
Processing	Persist a Notification row and push it over Flask-SocketIO to the recipient's personal room (user_<id>); the navbar bell count and a pop-up update instantly.
Output	Notification appears in the recipient's notification feed immediately, or on next login if offline.

FR-11 Direct Messaging (Chat)

FR-11	Direct Messaging
Description	Allow two authenticated users to exchange direct messages.
Input	Sender, receiver, message content
Processing	Validate the message (non-empty, ≤ 2000 characters); persist a Message row; push it over Flask-SocketIO to both participants' rooms.
Output	Message appears in both users' chat history.

4.5 Administration

FR-12 Manage Users

FR-12	Manage Users
Description	Allow an admin to create users, change their role, and deactivate or re-activate accounts.
Input	User details (name, email, role, phone, temporary password)
Processing	Validate and persist changes to the User table; a deactivated user cannot log in and any open session ends on the next request; an admin cannot deactivate themself; log the action (FR-15).
Output	User list reflects the change; an audit entry is recorded.

FR-13 Manage Logistics Hubs

FR-13	Manage Logistics Hubs
Description	Allow an admin to create, update, or delete hub master data.
Input	Hub name, location, latitude, longitude
Processing	Validate and persist changes to the Hub table; deletion is blocked while any shipment uses the hub.
Output	Hub list updated; hubs available for shipment origin/destination selection.

FR-14 Manage Cargo Types

FR-14	Manage Cargo Types
Description	Allow an admin to create, update, or delete cargo-type master data, including its sensitivity weight.
Input	Cargo type name, sensitivity_weight (0.0-1.0)
Processing	Validate the name is unique and the weight is between 0.0 and 1.0; persist changes to the CargoType table; deletion is blocked while any shipment uses the cargo type.
Output	Cargo type list updated; used as an input to FR-5 risk scoring.

FR-15 Activity Audit Log

FR-15	Activity Audit Log
Description	Record administrative and disruption-resolution actions for accountability.
Input	Acting user, action description, timestamp
Processing	Append an ActivityLog row whenever an admin or manager performs a tracked action.
Output	A chronological audit trail viewable by admins, filterable by date in Reports and exportable as CSV (FR-20).

4.6 AI Assistant & Live Map

FR-16 AI Assistant Query Handling

FR-16	AI Assistant Query Handling
Description	Answer a user's natural-language question about the supply chain.
Input	User's free-text question
Processing	Send the question, recent conversation and a snapshot of live data the user may see (shipments, risk, open disruptions and their ranked options) to Google Gemini; if no key is configured or the call fails, a rule-based offline engine answers from the same data.
Output	A natural-language answer in the assistant panel, labelled with its source (Gemini or offline engine).

FR-17 Live Map Visualisation

FR-17	Live Map Visualisation
Description	Display shipment positions and active disruption zones on an interactive map.
Input	All active shipments' current_lat/current_lng; open disruptions
Processing	Render a Leaflet.js map with markers coloured by risk_level, hubs, planned routes, and shaded circular zones around open disruptions sized by severity; refresh on every Socket.IO update.
Output	An interactive Live Map page available to every role (field operators see only their shipments).

FR-18 Demo / Simulation Mode

FR-18	Demo / Simulation Mode
Description	Let a user trigger a simulated disruption for demonstration purposes.
Input	A 'Trigger demo disruption' action (admin / manager); `flask reset-demo` to restore demo data
Processing	Inject a severe weather and congestion event (both 100) on an in-transit shipment, preferring a calm one, and run it through FR-5, FR-7 and FR-8 immediately.
Output	A live HIGH disruption with ranked options appears on the Disruptions page and the map, with real-time alerts, without waiting for the next scheduled scan.

4.7 Shipment Management & Reports

FR-19 Shipment Management

FR-19	Shipment Management
Description	Allow managers and admins to create, view, edit and delete shipments; let the assigned operator update status.
Input	Tracking no. (optional, auto CS-xxxx), origin, destination, mode, cargo type, value (₹), status, operator, position
Processing	Validate with Flask-WTF (origin ≠ destination, unique tracking no., valid ranges); default the position to the origin hub; run an initial risk scan; deleting a shipment also removes its disruption history.
Output	Shipment saved with an initial risk level; list, detail page (mini-map, disruption history) and live map update.

FR-20 Reports & CSV Export

FR-20	Reports & CSV Export
Description	Provide on-screen reports and CSV downloads for managers and admins.
Input	Report type, date range (from / to)
Processing	Aggregate disruptions by dominant factor and severity; compute resolution time (detection → decision) per decision-maker; list current shipment risk and the audit log; stream CSV with a UTF-8 BOM so Excel opens it correctly.
Output	Reports page with four reports; CSV files for disruptions, shipment risk and the activity log.

5. EXTERNAL INTERFACE REQUIREMENTS

5.1 User Interfaces

- Server-rendered Jinja2 templates styled with Bootstrap 5, sharing a common base layout (navbar, flash messages, footer).
- Role-specific dashboards: Admin (KPIs, four live charts, activity, tabs for Users / Shipments / Base Cities / Cargo Types / Reports / Config / Activity Log) and an operations dashboard for managers and field operators (KPIs, charts, risk watchlist, open disruptions).
- A Disruptions page presenting each open disruption with its ranked route solutions (time / cost / risk) and a one-click “Apply selected option”, plus Live Map, Chat, Notifications, AI Assistant and Reports pages.
- Responsive layout supporting desktop, tablet and mobile breakpoints via Bootstrap's grid system; front-end libraries are bundled locally so the interface works without internet (except map tiles).

5.2 Hardware Interfaces

- None required directly — the system runs on standard server hardware capable of hosting a Docker container.
- Shipment position data is simulated in this academic build in place of live GPS/telematics hardware (see Section 2.6).

5.3 Software Interfaces

Interface	Purpose	Fallback if Unavailable
OpenWeatherMap API	Supplies live weather severity for the risk engine (FR-5)	Simulated weather severity (mostly calm, occasional storm)
Google Gemini API	Powers the AI assistant's natural-language answers (FR-16)	Offline rule-based engine answers from the same live data; panel remains fully usable
SQLite	Primary data store, accessed via Flask-SQLAlchemy ORM	Not applicable — core dependency

5.4 Communication Interfaces

- HTTP/HTTPS for all standard page requests and form submissions.
- WebSocket (via Flask-SocketIO) for real-time notification and chat delivery.
- Outbound HTTPS calls to OpenWeatherMap and Gemini APIs when API keys are configured.

6. NON-FUNCTIONAL REQUIREMENTS

ID	Category	Requirement
NFR-1	Performance	Risk scores must refresh for all active shipments at least once every 60 seconds (configurable via RISK_SCAN_INTERVAL_SECONDS).
NFR-2	Security	Passwords must be hashed with Werkzeug's generate_password_hash — never stored or logged in plain text. All state-changing forms must include CSRF protection (Flask-WTF).
NFR-3	Reliability	Database writes must be transactional; a failed scheduled scan must not corrupt existing shipment data and must be retried on the next interval.
NFR-4	Usability	Each user role must see only the navigation and actions relevant to it; core workflows (login, view alerts, resolve disruption) must be completable within 3 clicks from the dashboard.
NFR-5	Portability	The application must run identically on any Docker-supported host via `docker compose up --build`, with no manual environment configuration beyond a .env file.
NFR-6	Maintainability	Code must be organised into independent Flask blueprints and a services/ package, so a feature (e.g. Route Solver) can be modified without touching unrelated modules.
NFR-7	Availability	The background scheduler and web server must both run inside the same container lifecycle, restarting together on failure (Docker `restart: unless-stopped`).
NFR-8	Scalability (future)	While SQLite suits this academic single-node build, the ORM-based data layer must allow migration to PostgreSQL/MySQL without rewriting business logic (see Proposed Enhancements, main documentation).

7. DATA REQUIREMENTS

The system persists nine entities in SQLite via the Flask-SQLAlchemy ORM. The core entities directly referenced by the functional requirements in Section 4 are detailed below; supporting entities (notifications, messages, hubs, cargo_types, activity_log) follow the same structure documented in the main project report.

users

Column Name	Data Type	Description
id	INTEGER (PK)	Unique user identifier
name / email	VARCHAR	Identity fields; email is unique
password_hash	VARCHAR(255)	Werkzeug-hashed password (FR-1, NFR-2)
role	VARCHAR(20)	admin / manager / captain / pilot / driver / loco_pilot (FR-4)
is_active	BOOLEAN	False = deactivated by an admin (FR-12)

shipments

Column Name	Data Type	Description
tracking_no	VARCHAR(30)	Human-readable tracking number (FR-19)
status	VARCHAR(20)	In-transit / Delayed / On-hold / Delivered
risk_score / risk_level	FLOAT / VARCHAR	Output of FR-5 / FR-6 (LOW / AT-RISK / MEDIUM / HIGH)
current_lat / current_lng	FLOAT	Position used by FR-6 movement and FR-17 live map
cargo_value	FLOAT	Declared value in INR (reports, value at risk)
assigned_operator_id	INTEGER (FK)	Field operator responsible (FR-4, FR-9, FR-10)

disruptions / route_solutions

Column Name	Data Type	Description
disruptions.risk_score / severity / status	mixed	Output of FR-7
route_solutions.extra_time_hours / extra_cost / risk_score	FLOAT	Output of FR-8, input to FR-9
route_solutions.selected	BOOLEAN	Set by FR-9 disruption resolution

8. APPENDIX — TRACEABILITY & BIBLIOGRAPHY

8.1 Requirements Traceability Summary

Maps each functional requirement to the codebase module that implements it and the test case that verifies it (test cases are in Section 5 of the main project report).

Requirement	Implementing Module	Status / Test
FR-1 to FR-4	app/blueprints/auth/, role checks in every blueprint	Implemented — TC001, TC007
FR-5, FR-6	app/services/risk_engine.py, APScheduler job in app/__init__.py	Implemented — TC002, TC003
FR-7, FR-8, FR-9	app/services/risk_engine.py, app/services/route_solver.py, app/blueprints/disruptions/	Implemented — TC003, TC004
FR-10, FR-11	app/services/notifier.py, app/sockets.py, app/blueprints/notifications/, app/blueprints/chat/	Implemented — TC004, TC008
FR-12 to FR-15	app/blueprints/main/ (admin tabs), ActivityLog model	Implemented — TC005, TC011
FR-16	app/services/assistant.py, app/blueprints/assistant/	Implemented — TC009
FR-17	app/blueprints/shipments/ (live map + JSON feed), Leaflet.js	Implemented — TC012
FR-18	trigger_demo_disruption() in risk_engine.py; flask reset-demo	Implemented — TC012
FR-19	app/blueprints/shipments/, app/forms.py	Implemented — automated CRUD checks
FR-20	Reports routes in app/blueprints/main/	Implemented — TC010

8.2 Bibliography

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- Flask Web Development — Miguel Grinberg, O'Reilly Media
- Fundamentals of Database Systems — Ramez Elmasri & Shamkant B. Navathe, Pearson
- https://flask.palletsprojects.com/
- https://docs.docker.com/
- https://www.sqlite.org/docs.html
- https://flask-socketio.readthedocs.io/
- https://leafletjs.com/reference.html
- https://ai.google.dev/gemini-api/docs